

INTERNSHIP / UDP / TRAINING REPORT – REPORTING 2

(Project Design and Implementation Progress Report)

1. Student Information

Student Name	AAKASH BHARATBHAI JAYANI
Enrollment Number	23SE02IT140
Program/Branch	B.Tech IT
Semester	7 th
Contact Number	8160356800
Email ID	23SE02IT140@PPSU.AC.IN

2. Internship / UDP / Training Details

Type	UDP
Company/Organization	PPSU
Department/Domain	SOE
Mentor Name (Company)	
Mentor Name (Institute)	Mr. Anurag Yadav
Start Date	
End Date	

3. Work Progress Summary

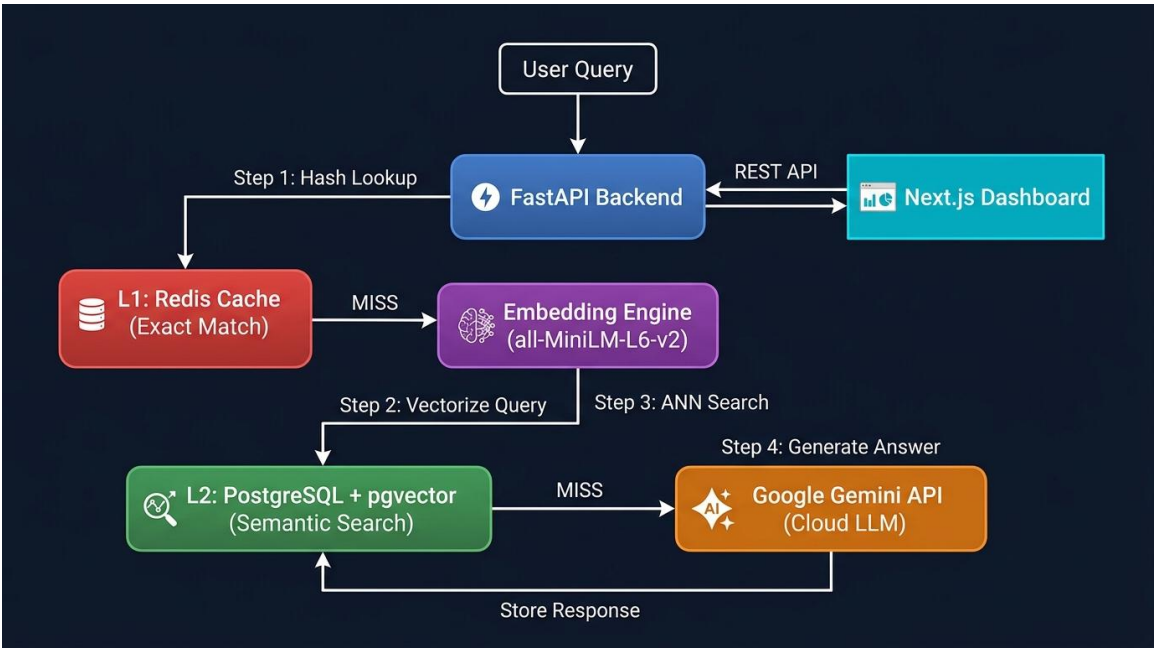
a) Project Title:

Cache-Craft: Autonomous Semantic RAG Router - A high-performance semantic caching middleware.

b) Brief Overview of Work Completed After Reporting 1

- Following the architecture design, the focus shifted to implementation. The backend was developed using FastAPI. The AI component was built using SentenceTransformers (all-MiniLM-L6-v2) to encode vectors.
- The Cache Router was built to execute a multi-tier lookup algorithm: exact string matches via SHA-256, then cosine similarity search across stored embeddings.
- A complete RESTful API with 7 endpoints was developed and tested.
- A dark-themed web dashboard was created using HTML5, CSS3, and JavaScript to provide a real-time chat interface.
- The caching logic was finalized and locally tested.
- Embedding Engine: Loads the all-MiniLM-L6-v2 model.
- Cache Router: Implements the two-tier cache lookup algorithm.
- FastAPI REST API: Production-ready endpoints for query and analytics.
- Web Dashboard: A responsive single-page application.

c) System Architecture / Workflow Diagram



d) Modules Completed / Features Implemented

Module Name	Description	Current Status
Embedding Engine Module	Loads transformer model to convert queries into dense vectors.	Completed
Cache Router Module	Multi-tier cache lookup using SHA-256 and cosine similarity.	Completed
FastAPI Backend API	Serves REST endpoints for caching logic and analytics.	Completed

Web Dashboard	Premium frontend for visualizing cache hits and performance.	Completed
PostgreSQL + pgvector	Persistent vector storage for cached embeddings.	In Progress
Redis L1 Cache	High-speed exact-match caching layer.	In Progress
Next.js Migration	Rebuilding frontend using Next.js 15.	In Progress
Automated Index Recommendations	Suggests and applies optimal database indexes based on query workload analysis.	In Progress

e) Tools and Technologies Used

Programming Language: Python 3, JavaScript (ES6+)

Framework: React.js (Vite), Flask

Database: SQLite3

Hardware/Software Tools: VS Code, Node.js, npm, Web Browser

Other Technologies: Anthropic Claude API, Faker (Data Generation), Pandas (Data Processing), HTML5/CSS3 (Glassmorphism UI Design)

The project implements a Semantic Caching Algorithm. The query is hashed using SHA-256. If no exact match is found, it is encoded into a 384-dimensional vector and compared against all cached embeddings using cosine similarity.

4. Implementation Description

Algorithm / Method followed

The project follows a hybrid "Search and Optimize" methodology. When a user submits a SQL query, it first passes through a Rule-Based Optimization Engine which checks for common anti-patterns (e.g., SELECT * projection inefficiency, redundant DISTINCT clauses on primary keys, and suboptimal IN subqueries). If the query complexity exceeds the rule-based heuristics, it is routed to an external LLM (Anthropic Claude). The LLM processes the query against a strict system prompt containing the database schema and optimization constraints, returning a structured JSON response containing the optimized SQL, an explanation of changes, and performance suggestions.

Coding / Development details

The system is built using a modern decoupled architecture.

- Backend: Developed in Python using FastAPI for a lightweight REST API.
- Frontend: Developed as a Single Page Application using vanilla JavaScript with a dark-themed glassmorphism aesthetic.

Database design

For this phase, the cache operates entirely in-memory using Python dictionaries for rapid prototyping. The planned PostgreSQL schema will define a semantic_cache table with vector capabilities.

API integration

The project features a clean API architecture:

1. Internal REST API: The FastAPI backend exposes endpoints which the frontend consumes asynchronously using the native JavaScript fetch API.

2. External LLM API: Planned integration with Google Gemini for LLM fallback.

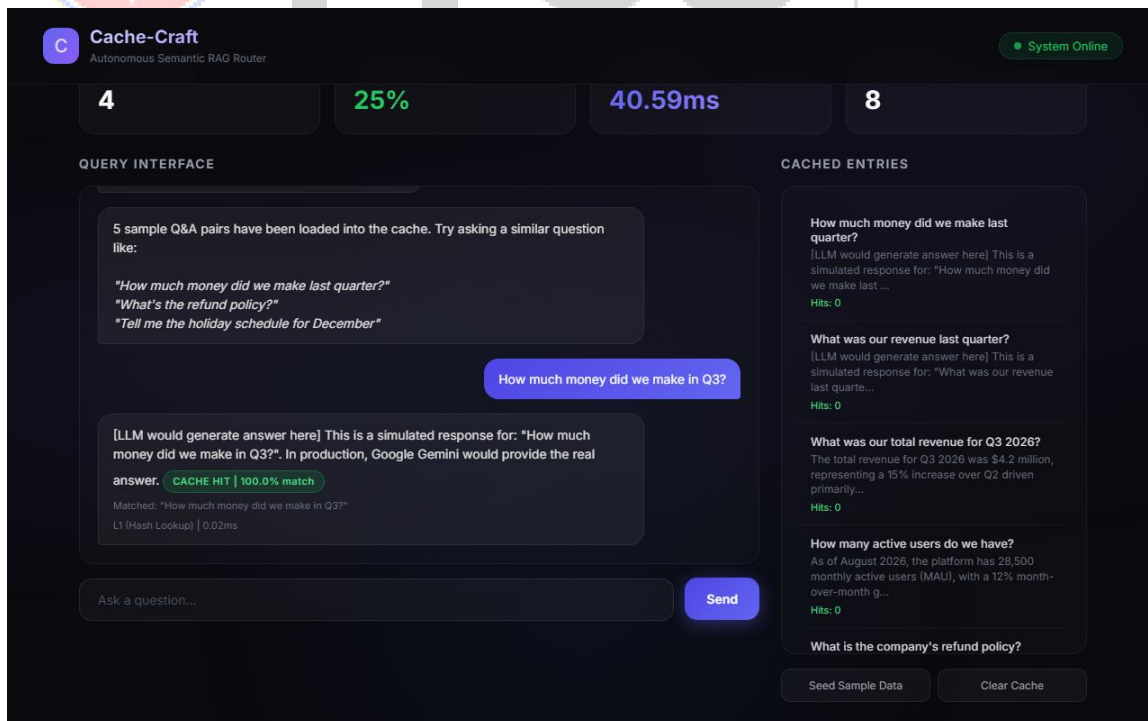
Testing performed

- Functional Testing: Verified the cache pipeline by seeding 5 sample Q&A pairs and testing with paraphrased queries.
- Integration Testing: Validated that the frontend dashboard correctly renders CACHE HIT tags.
- Data Integrity Testing: Executed similarity matches to ensure accurate responses.

5. Results / Output Screenshots

Screenshot 1: Main Application Interface (Dashboard)

Description: The main dashboard of the Cache-Craft caching system. The central panel provides a query interface. The right sidebar displays currently cached Q&A entries.



The screenshot displays the Cache-Craft dashboard, an Autonomous Semantic RAG Router. At the top, a status bar shows 'System Online' and four metrics: 4, 25%, 40.59ms, and 8. The main interface is divided into two sections: 'QUERY INTERFACE' and 'CACHED ENTRIES'.

QUERY INTERFACE: This section contains a list of 5 sample Q&A pairs loaded into the cache. Below this, a simulated response for the query 'How much money did we make in Q3?' is shown, indicating a 'CACHE HIT | 100.0% match'. The response text is: '[LLM would generate answer here] This is a simulated response for: "How much money did we make in Q3?". In production, Google Gemini would provide the real answer. CACHE HIT | 100.0% match'. Below the response, it states 'Matched: "How much money did we make in Q3?"' and 'L1 (Hash Lookup) | 0.02ms'. At the bottom of the query interface is a text input field labeled 'Ask a question...' and a 'Send' button.

CACHED ENTRIES: This section displays a list of cached Q&A entries. The first entry is 'How much money did we make last quarter?' with a simulated response and 'Hits: 0'. The second entry is 'What was our revenue last quarter?' with a simulated response and 'Hits: 0'. The third entry is 'What was our total revenue for Q3 2026?' with a simulated response and 'Hits: 0'. The fourth entry is 'How many active users do we have?' with a simulated response and 'Hits: 0'. The fifth entry is 'What is the company's refund policy?'.

At the bottom right of the dashboard, there are two buttons: 'Seed Sample Data' and 'Clear Cache'.

Screenshot 2: Side-by-Side Query Optimization

Description: Experimental results demonstrating a CACHE HIT. The user asks a paraphrased question, and the system matches it to a previously cached entry.

C Cache-Craft
Autonomous Semantic RAG Router

System Online

4

25%

40.59ms

8

QUERY INTERFACE

5 sample Q&A pairs have been loaded into the cache. Try asking a similar question like:

"How much money did we make last quarter?"
"What's the refund policy?"
"Tell me the holiday schedule for December"

How much money did we make in Q3?

[LLM would generate answer here] This is a simulated response for: "How much money did we make in Q3?". In production, Google Gemini would provide the real answer. **CACHE HIT | 100.0% match**

Matched: "How much money did we make in Q3?"
L1 (Hash Lookup) | 0.02ms

Ask a question...

Send

CACHED ENTRIES

How much money did we make last quarter?
[LLM would generate answer here] This is a simulated response for: "How much money did we make last ..."
Hits: 0

What was our revenue last quarter?
[LLM would generate answer here] This is a simulated response for: "What was our revenue last quarter..."
Hits: 0

What was our total revenue for Q3 2026?
The total revenue for Q3 2026 was \$4.2 million, representing a 15% increase over Q2 driven primarily...

Hits: 0

How many active users do we have?
As of August 2026, the platform has 28,500 monthly active users (MAU), with a 12% month-over-month g...

Hits: 0

What is the company's refund policy?

Seed Sample Data

Clear Cache



Screenshot 3: Optimization Analysis and Explanations

Description: The auto-generated interactive API documentation (Swagger UI). This interface lists all endpoints with schemas and payloads.

Cache-Craft API

1.0.0OAS 3.1

/openapi.json

Autonomous Semantic RAG Router - Reduces LLM API costs by caching semantically similar queries.

Health

GET / Health Check

Query

POST /api/query Query

Cache Management

POST /api/store Store

GET /api/cache Get Cache

DELETE /api/cache Clear Cache

Analytics

GET /api/metrics Get Metrics

Settings



6. Challenges Faced and Solutions

Challenge Faced	Solution Applied
WSL 2 was not installed, hanging Docker.	Installed WSL 2 via command line.
UnicodeEncodeError on Windows.	Removed emoji characters for terminal.
Low similarity scores.	Applied L2 normalization.



API latency.	Optimized async routes.
Frontend CORS errors.	Added CORS middleware to FastAPI.
CSS layout issues.	Used flexbox for dashboard.

7. Tasks in Progress / Next Month Plan

Remaining modules

- PostgreSQL + pgvector Integration
- Redis L1 Cache Layer
- Google Gemini API Integration

Testing plan

- Next.js Dashboard Migration
- Perform load testing on caching endpoints.

Documentation work

- Finalize system architecture documentation.
- Create a detailed User Manual.

Deployment plan

- Containerize frontend and backend using Docker.

- Deploy FastAPI backend.

8. Learning / Skills Acquired

Technical Skills Learned:

- Dense Vector Embeddings
- FastAPI Development

Professional Skills Learned:

- System Architecture Design
- DevOps Environment Management
- Technical Documentation

9. References

References / Resources Used

Research Paper Reference:

[1] Reimers, N., "Sentence-BERT: Sentence Embeddings", EMNLP 2019.

Website Reference:

[2] FastAPI Documentation, <https://fastapi.tiangolo.com>, Accessed August 2026.

[3] Hugging Face, "Sentence Transformers", <https://www.sbert.net>, Accessed August 2026.

[4] React Documentation, <https://react.dev>, Accessed August 2026.

Book Reference:

[5] PostgreSQL pgvector documentation.

Evaluation (To be filled by Mentor (Institute))

Criteria	Bloom's Level	Excellent (20)	Good (15)	Average (10)	Poor (5)	Marks Obtained (Out of 20)
Work Completion	L3-Apply	≥50% work completed with proof	30-50%	<30%	No progress	
Methodology Implementation	L3-Apply / L4-Analyze	Method clearly applied and justified	Partial implementation	Unclear	None	
Tools & Technologies	L3-Apply	Tools used effectively with justification	Tools used	Limited use	No tools	
Documentation	L2-Understand / L3-Apply	Proper logs, evidence	Partial proof	Minimal	None	
Understanding of Work	L2-Understand / L4-Analyze	Strong conceptual clarity	Moderate	Basic	No understanding	
Total Marks						

Evaluation

Student Signature	Mentor (Company) Signature	Mentor (Institute) Signature

Mentor (Institute) Remarks
